CIFAR-10 Neural Network Classification — Results Report

Jieyao chen

Foundation of AI Homework 3

Network 1: FeedForwardNet

Architecture Description

FeedForwardNet is a fully connected (feed-forward) network with three hidden layers. The input image (32×32×3) is first flattened into a vector of 3,072 values. This vector is passed through three hidden layers of sizes 1,024, 512, and 256 neurons respectively, each followed by a ReLU activation function. The final output layer maps to 10 classes with no activation (raw logits for CrossEntropyLoss). No convolutional, pooling, or dropout layers are used.

Layer summary: Flatten → Linear(3072→1024) → ReLU → Linear(1024→512) → ReLU → Linear(512→256) → ReLU → Linear(256→10)

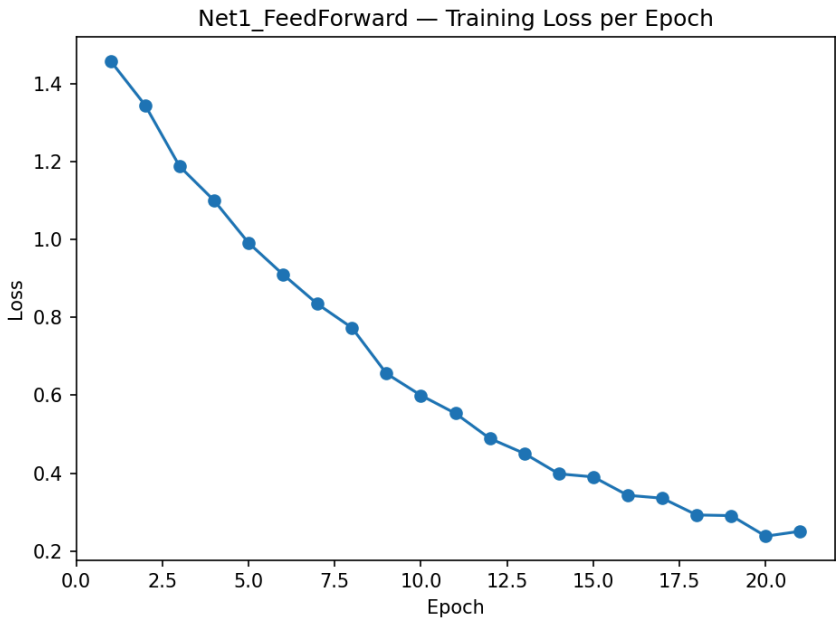
Test Accuracy

54.83%

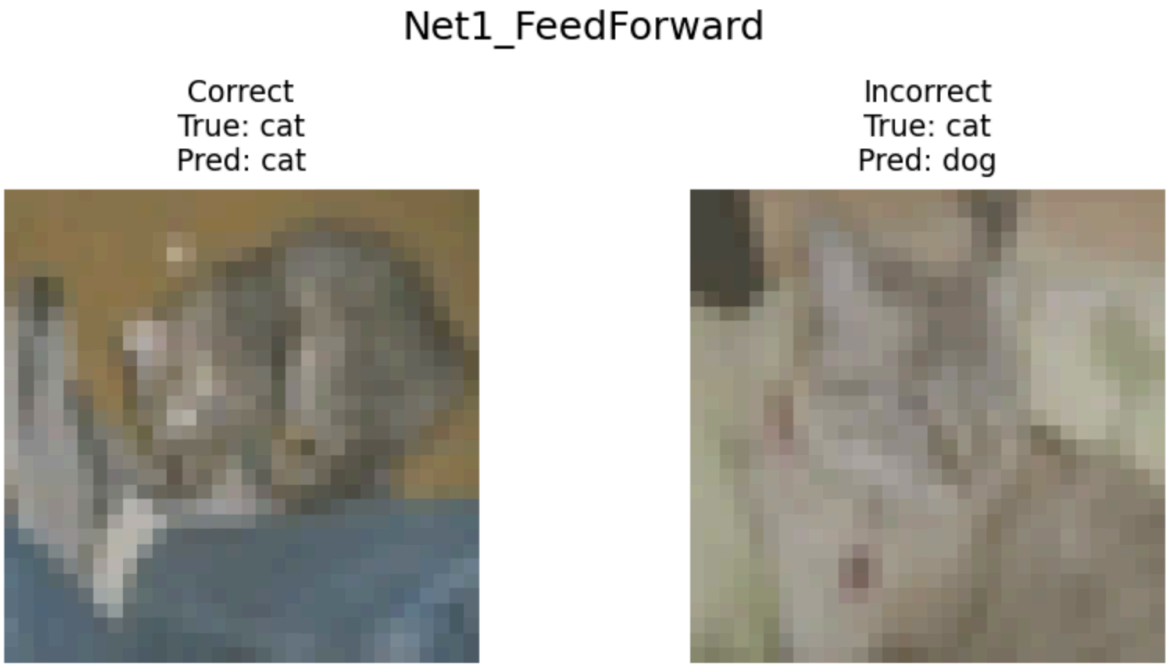
Training stopped early at epoch 21 (loss increased from 0.2382 to 0.2510).

```
================================================
 Training Net1_FeedForward
================================================
[Net1_FeedForward] Epoch  1  train loss: 1.4576
[Net1_FeedForward] Epoch  2  train loss: 1.3444
[Net1_FeedForward] Epoch  3  train loss: 1.1874
[Net1_FeedForward] Epoch  4  train loss: 1.0998
[Net1_FeedForward] Epoch  5  train loss: 0.9910
[Net1_FeedForward] Epoch  6  train loss: 0.9099
[Net1_FeedForward] Epoch  7  train loss: 0.8341
[Net1_FeedForward] Epoch  8  train loss: 0.7731
[Net1_FeedForward] Epoch  9  train loss: 0.6556
[Net1_FeedForward] Epoch 10  train loss: 0.5997
[Net1_FeedForward] Epoch 11  train loss: 0.5535
[Net1_FeedForward] Epoch 12  train loss: 0.4890
[Net1_FeedForward] Epoch 13  train loss: 0.4506
[Net1_FeedForward] Epoch 14  train loss: 0.3983
[Net1_FeedForward] Epoch 15  train loss: 0.3904
[Net1_FeedForward] Epoch 16  train loss: 0.3432
[Net1_FeedForward] Epoch 17  train loss: 0.3358
[Net1_FeedForward] Epoch 18  train loss: 0.2927
[Net1_FeedForward] Epoch 19  train loss: 0.2910
[Net1_FeedForward] Epoch 20  train loss: 0.2382
[Net1_FeedForward] Epoch 21  train loss: 0.2510
[Net1_FeedForward] Loss increased — stopping early at epoch 21.
Saved Net1_FeedForward_loss.png
[Net1_FeedForward] Test accuracy: 54.83%
```

Training Loss Curve



Classification Examples



Network 2: SmallCNN

Architecture Description

SmallCNN is a convolutional neural network with two convolutional layers and two fully connected layers. The network is divided into a feature extractor and a classifier. The feature extractor applies a Conv2d layer (3→32 channels, 3×3 kernel, padding=1) followed by ReLU and MaxPool2d(2×2), reducing the spatial size from 32×32 to 16×16. A second Conv2d layer (32→64 channels) is then applied, again followed by ReLU and MaxPool2d(2×2), reducing size to 8×8. The resulting feature map (64×8×8 = 4,096 values) is flattened and passed through two fully connected layers (4096→512→10).

Layer summary: Conv2d(3→32) → ReLU → MaxPool2d → Conv2d(32→64) → ReLU → MaxPool2d → Flatten → Linear(4096→512) → ReLU → Linear(512→10)

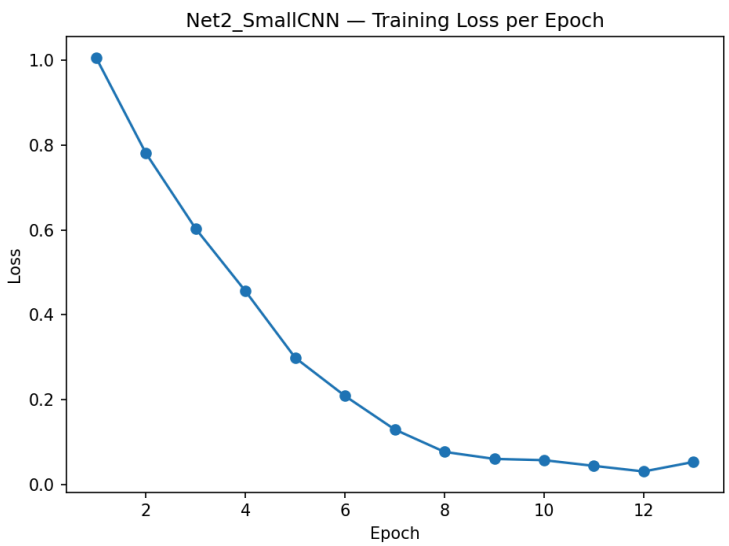
Test Accuracy

71.52%

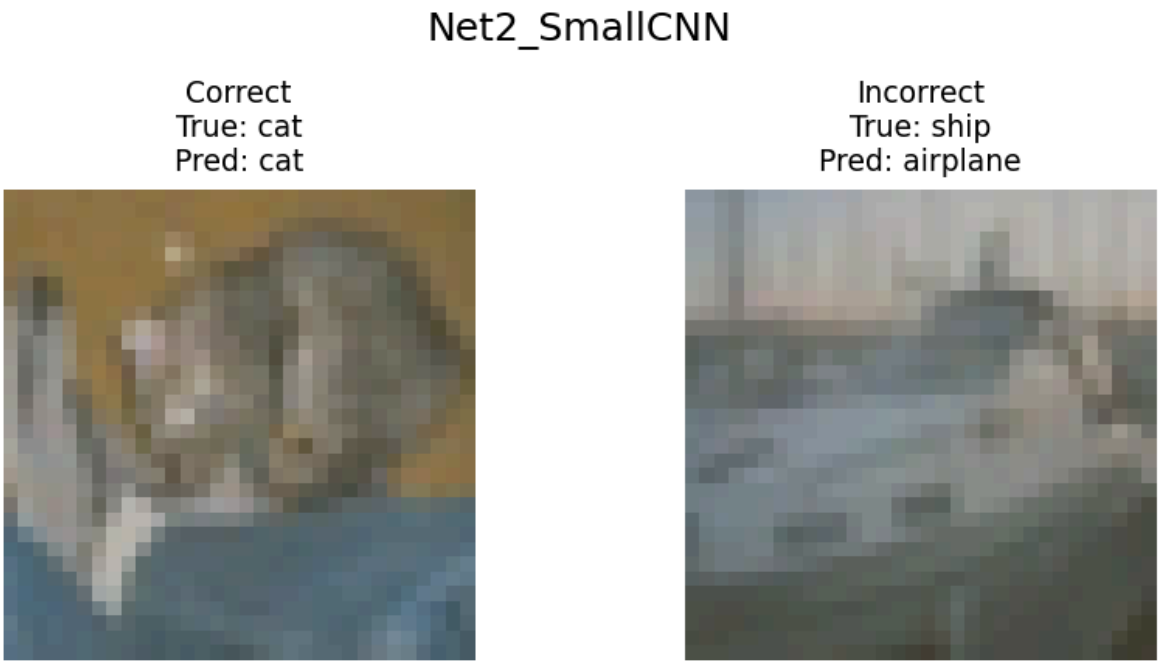
```
==================================================
 Training Net2_SmallCNN
==================================================
[Net2_SmallCNN] Epoch  1  train loss: 1.0068
[Net2_SmallCNN] Epoch  2  train loss: 0.7814
[Net2_SmallCNN] Epoch  3  train loss: 0.6034
[Net2_SmallCNN] Epoch  4  train loss: 0.4559
[Net2_SmallCNN] Epoch  5  train loss: 0.2987
[Net2_SmallCNN] Epoch  6  train loss: 0.2086
[Net2_SmallCNN] Epoch  7  train loss: 0.1294
[Net2_SmallCNN] Epoch  8  train loss: 0.0766
[Net2_SmallCNN] Epoch  9  train loss: 0.0601
[Net2_SmallCNN] Epoch 10  train loss: 0.0570
[Net2_SmallCNN] Epoch 11  train loss: 0.0436
[Net2_SmallCNN] Epoch 12  train loss: 0.0304
[Net2_SmallCNN] Epoch 13  train loss: 0.0529
[Net2_SmallCNN] Loss increased — stopping early at epoch 13.
Saved Net2_SmallCNN_loss.png
[Net2_SmallCNN] Test accuracy: 71.52%
Saved Net2_SmallCNN_examples.png
```

Training stopped early at epoch 13 (loss increased from 0.0304 to 0.0529).

Training Loss Curve



Classification Examples



Network 3: DeepCNN

Architecture Description

DeepCNN is a deeper convolutional network with three convolutional layers, two pooling layers, and a dropout layer for regularization. The feature extractor stacks two Conv2d layers (3→32 and 32→64) before the first MaxPool2d, allowing the network to learn richer low-level features before downsampling. A third Conv2d layer (64→128 channels) is then applied, followed by a second MaxPool2d, yielding a feature map of size 128×8×8. In the classifier, a Dropout layer (p=0.5) is applied before the first fully connected layer (8192→256) to reduce overfitting. The final output layer maps to 10 classes.

Layer summary: Conv2d(3→32) → ReLU → Conv2d(32→64) → ReLU → MaxPool2d → Conv2d(64→128) → ReLU → MaxPool2d → Flatten → Dropout(0.5) → Linear(8192→256) → ReLU → Linear(256→10)

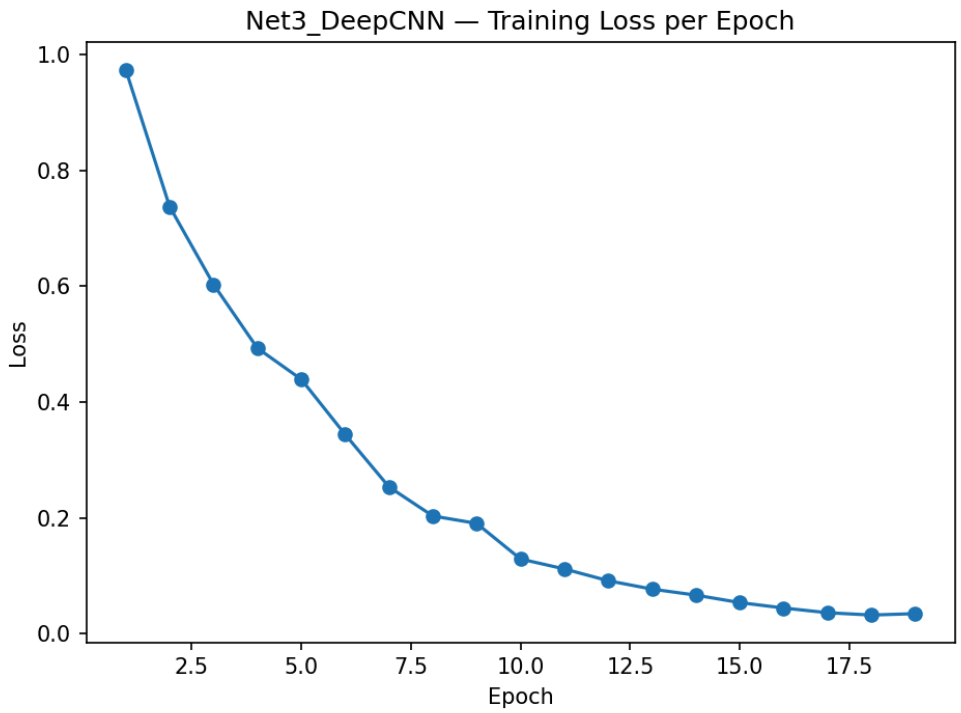
Test Accuracy

77.76%

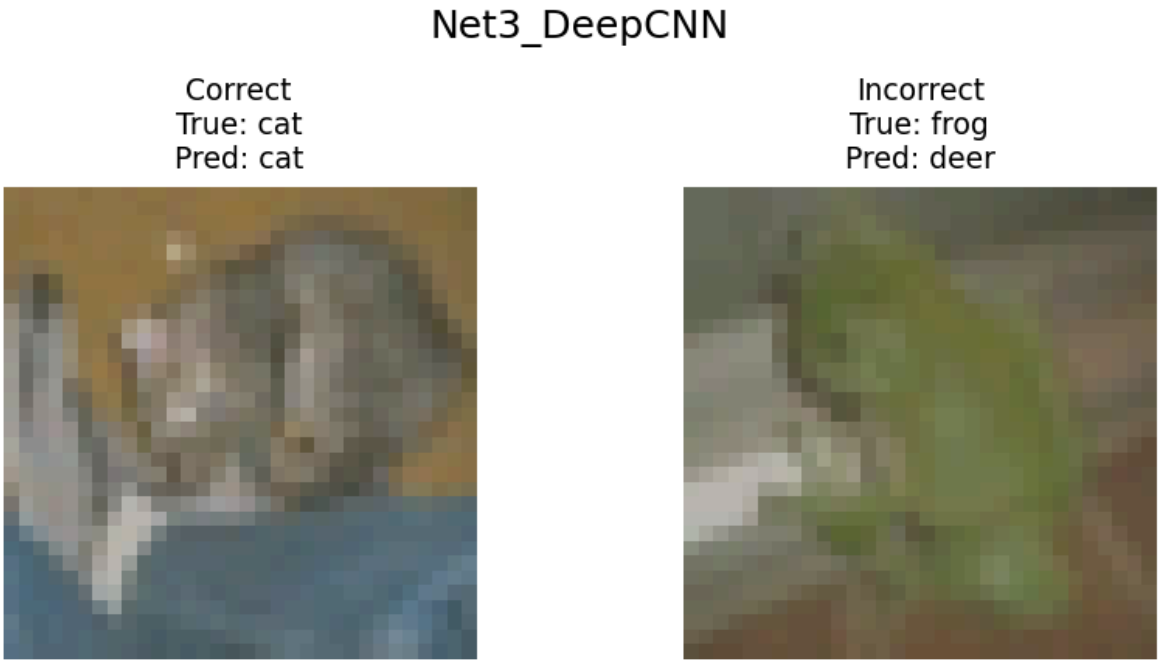
```
==================================================
 Training Net3_DeepCNN
==================================================
[Net3_DeepCNN] Epoch  1  train loss: 0.9735
[Net3_DeepCNN] Epoch  2  train loss: 0.7372
[Net3_DeepCNN] Epoch  3  train loss: 0.6031
[Net3_DeepCNN] Epoch  4  train loss: 0.4931
[Net3_DeepCNN] Epoch  5  train loss: 0.4393
[Net3_DeepCNN] Epoch  6  train loss: 0.3442
[Net3_DeepCNN] Epoch  7  train loss: 0.2536
[Net3_DeepCNN] Epoch  8  train loss: 0.2034
[Net3_DeepCNN] Epoch  9  train loss: 0.1905
[Net3_DeepCNN] Epoch 10  train loss: 0.1291
[Net3_DeepCNN] Epoch 11  train loss: 0.1119
[Net3_DeepCNN] Epoch 12  train loss: 0.0916
[Net3_DeepCNN] Epoch 13  train loss: 0.0771
[Net3_DeepCNN] Epoch 14  train loss: 0.0667
[Net3_DeepCNN] Epoch 15  train loss: 0.0538
[Net3_DeepCNN] Epoch 16  train loss: 0.0445
[Net3_DeepCNN] Epoch 17  train loss: 0.0364
[Net3_DeepCNN] Epoch 18  train loss: 0.0324
[Net3_DeepCNN] Epoch 19  train loss: 0.0347
[Net3_DeepCNN] Loss increased — stopping early at epoch 19.
Saved Net3_DeepCNN_loss.png
[Net3_DeepCNN] Test accuracy: 77.76%
Saved Net3_DeepCNN_examples.png
```

Training stopped early at epoch 19 (loss increased from 0.0324 to 0.0347).

Training Loss Curve



Classification Examples



Comparison and Analysis

Summary of Results

Network	Architecture	Epochs	Final Train Loss	Test Accuracy
FeedForwardNet	Fully Connected	21	0.2382	54.83%
SmallCNN	2-layer CNN	13	0.0304	71.52%
DeepCNN	3-layer CNN+Dropout	19	0.0324	77.76%

Best Network: DeepCNN (77.76%)

DeepCNN achieved the highest test accuracy for several reasons. First, its three convolutional layers allow it to learn increasingly abstract features — from low-level edges in early layers to complex shapes and textures in deeper layers — which are more discriminative for classification. Second, the additional Conv2d(32→64) layer before the first pooling operation preserves more spatial detail before downsampling, giving the network richer information to work with. Most importantly, the Dropout(0.5) layer in the classifier acts as a regularizer: by randomly disabling 50% of neurons during training, it forces the network to learn more robust, generalized features rather than memorizing specific training examples. This is reflected in the fact that DeepCNN's **final train loss (0.032) is significantly lower than FeedForwardNet's (0.238)**, yet its test accuracy is substantially higher (77.76% vs 54.83%), indicating that Dropout effectively reduced overfitting and improved generalization.

Worst Network: FeedForwardNet (54.83%)

FeedForwardNet performed the worst for two main reasons. First, flattening the image into a 1D vector at the very first step destroys all spatial information — the network has no way to know that two pixels are adjacent, which is critical for image recognition tasks. Second, the fully connected architecture has an extremely large number of parameters (the first layer alone has 3,072 × 1,024 = 3,145,728 weights), making it highly prone to overfitting. This is evident from the gap between its final train loss (0.238) and its test accuracy (54.83%): the model learned to fit the training data but failed to generalize. Without dropout or convolutional weight sharing, there is little to prevent memorization.

Key Takeaways

The results clearly demonstrate the advantage of convolutional architectures for image classification tasks. CNNs leverage spatial locality and weight sharing to learn meaningful visual features efficiently, while feed-forward networks treat images as flat vectors and struggle to generalize. Furthermore, regularization techniques such as Dropout meaningfully improve generalization, as shown by the gap between SmallCNN (71.52%) and DeepCNN (77.76%) despite both being CNN-based architectures.